

PROJECT REPORT

HOSTEL MANAGEMENT SYSTEM

STUDENT & COURSE DETAILS

- **Student Name:** Mehul Vashisth
- **Registration Number:** 25BAI11510
- **Course Title:** Programming in Java
- **Faculty In-Charge:** Ms. Murugeswari K.
- **Academic Year:** 2025–2026

1. ABSTRACT

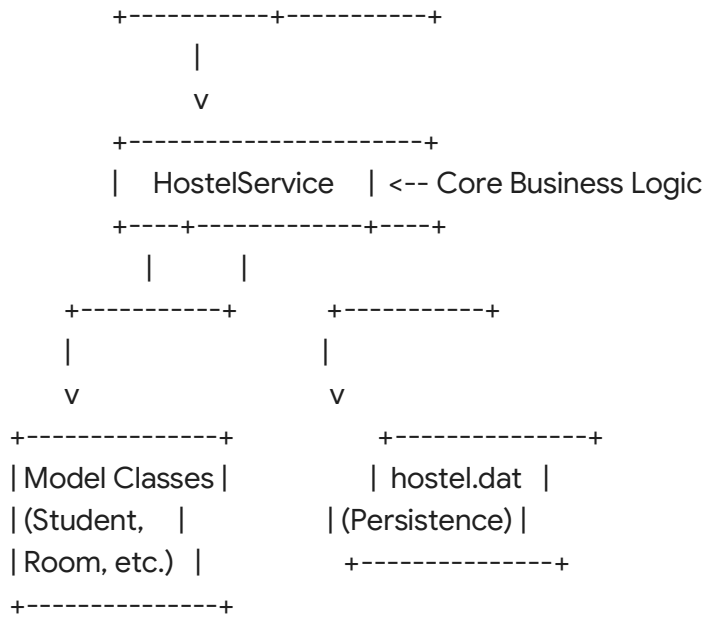
The **Hostel Management System** is a software application designed to streamline and automate day-to-day administrative operations within a residential establishment. Built using Java, the application provides an interactive console-based interface for managing student registrations, room allocations, complaint logging, fee collection tracking, and analytics generation. By utilizing object serialization, the system provides reliable data persistence (data/hostel.dat), ensuring operational continuity across application sessions.

2. SYSTEM ARCHITECTURE & MODULES

The application is structured into four distinct modular packages, adhering to clear separation of concerns and standard Object-Oriented Programming (OOP) design patterns:

1. **app Package (Main.java):**
 - Serves as the entry point and presentation layer of the application.
 - Handles user interaction via a Command-Line Interface (CLI), interactive menus, input reading, and global exception catching.
2. **model Package:**
 - Encapsulates domain model objects, including Student, Room, Complaint, and Payment.
 - Houses basic data encapsulation, accessors, and state definitions.
3. **service Package (HostelService.java):**
 - Implements the core business logic, including room validation, bed allocation limits, status tracking, payment handling, and file persistence (load and save).
4. **exception Package:**
 - Contains custom runtime exceptions for handling logical errors (e.g., room capacity exceeded, duplicate IDs, invalid actions).

```
+-----+
| Main.java | <-- Terminal Interface
```



3. CORE FEATURES & FUNCTIONALITIES

The system offers 11 key operational commands accessible via the interactive console menu:

Option	Feature	Description
1	Add Student	Registers student personal details including ID, Name, Contact Phone, Course, and Academic Year.
2	Add Room	Creates room entries defined by unique Room Numbers and Bed Capacity limits.
3	Allot Room	Assigns an unallocated student to an available room, enforcing capacity checks.
4	Vacate Room	De-allocates a student from their assigned room

		and frees up capacity.
5	List Students	Renders detailed list profiles of all registered students.
6	List Rooms	Displays all room profiles, occupancy counts, and current occupants.
7	Add Complaint	Enables registered students to log grievance tickets.
8	Resolve Complaint	Updates ticket resolution status for logged complaints.
9	Record Payment	Processes and records hostel fee payments against student accounts.
10	Generate Report	Computes aggregate metrics including total students, active rooms, occupied beds, open complaints, and revenue summary.
0	Save & Exit	Serializes system state to disk (data/hostel.dat) and terminates the program safely.

4. SOURCE CODE IMPLEMENTATION

Main.java

```
package app;

import model.*;
```



```

        break;
    case "9":
        recordPayment(hostelService);
        break;
    case "10":
        generateReport(hostelService);
        break;
    case "0":
        saveData(hostelService);
        return;
    default:
        System.out.println("Invalid choice");
    }
} catch (Exception error) {
    System.out.println("Error: " + error.getMessage());
}
}
}

```

```

static void displayMenu() {
    System.out.println();
    System.out.println("===== HOSTEL MANAGEMENT SYSTEM =====");
    System.out.println("1. Add student");
    System.out.println("2. Add room");
    System.out.println("3. Allot room");
    System.out.println("4. Vacate room");
    System.out.println("5. List students");
    System.out.println("6. List rooms");
    System.out.println("7. Add complaint");
    System.out.println("8. Resolve complaint");
    System.out.println("9. Record payment");
    System.out.println("10. Generate report");
    System.out.println("0. Save & exit");
    System.out.print("Choose: ");
}

```

```

static void addStudent(HostelService hostelService) throws Exception {
    System.out.print("ID: ");
    int studentId = Integer.parseInt(scanner.nextLine());

    System.out.print("Name: ");
}

```

```

String name = scanner.nextLine();

System.out.print("Phone: ");
String phone = scanner.nextLine();

System.out.print("Course: ");
String course = scanner.nextLine();

System.out.print("Year: ");
int year = Integer.parseInt(scanner.nextLine());

Student student = new Student(studentId, name, phone, course, year);
hostelService.addStudent(student);
}

static void addRoom(HostelService hostelService) throws Exception {
    System.out.print("Room no: ");
    int roomNumber = Integer.parseInt(scanner.nextLine());

    System.out.print("Capacity: ");
    int capacity = Integer.parseInt(scanner.nextLine());

    hostelService.addRoom(roomNumber, capacity);
}

static void allotRoom(HostelService hostelService) throws Exception {
    System.out.print("Student ID: ");
    int studentId = Integer.parseInt(scanner.nextLine());

    System.out.print("Room no: ");
    int roomNumber = Integer.parseInt(scanner.nextLine());

    hostelService.allotRoom(studentId, roomNumber);
}

static void vacateRoom(HostelService hostelService) throws Exception {
    System.out.print("Student ID: ");
    int studentId = Integer.parseInt(scanner.nextLine());

    hostelService.vacateRoom(studentId);
}

```

```
static void listStudents(HostelService hostelService) {  
    hostelService.students().forEach(System.out::println);  
}
```

```
static void listRooms(HostelService hostelService) {  
    hostelService.rooms().forEach(System.out::println);  
}
```

```
static void addComplaint(HostelService hostelService) throws Exception {  
    System.out.print("Student ID: ");  
    int studentId = Integer.parseInt(scanner.nextLine());  
  
    System.out.print("Complaint: ");  
    String complaintText = scanner.nextLine();  
  
    Complaint complaint = hostelService.addComplaint(studentId, complaintText);  
    System.out.println("Complaint added successfully. Complaint ID: " + complaint.getId());  
}
```

```
static void resolveComplaint(HostelService hostelService) throws Exception {  
    System.out.print("Complaint ID: ");  
    int complaintId = Integer.parseInt(scanner.nextLine());  
  
    hostelService.resolveComplaint(complaintId);  
}
```

```
static void recordPayment(HostelService hostelService) throws Exception {  
    System.out.print("Student ID: ");  
    int studentId = Integer.parseInt(scanner.nextLine());  
  
    System.out.print("Amount: ");  
    double amount = Double.parseDouble(scanner.nextLine());  
  
    hostelService.pay(studentId, amount);  
}
```

```
static void generateReport(HostelService hostelService) {  
    System.out.println("Students: " + hostelService.students().size());  
    System.out.println("Rooms: " + hostelService.rooms().size());  
}
```

```

        long occupiedBeds = hostelService.rooms()
            .stream()
            .mapToLong(room -> room.getOccupants().size())
            .sum();
        System.out.println("Occupied beds: " + occupiedBeds);

        System.out.println("Complaints: " + hostelService.complaints().size());

        double totalPayments = hostelService.payments()
            .values()
            .stream()
            .mapToDouble(Double::doubleValue)
            .sum();
        System.out.println("Payments: " + totalPayments);

        hostelService.complaints().forEach(System.out::println);
    }

    static HostelService loadHostelData() {
        try {
            return HostelService.load(DATA_FILE);
        } catch (Exception error) {
            return new HostelService();
        }
    }

    static void saveData(HostelService hostelService) {
        try {
            new File("data").mkdirs();
            hostelService.save(DATA_FILE);
            System.out.println("Saved successfully.");
        } catch (IOException error) {
            System.out.println("Could not save data.");
        }
    }
}

```

5. SAMPLE OUTPUT & CONSOLE EXECUTION

===== HOSTEL MANAGEMENT SYSTEM =====

1. Add student
2. Add room
3. Allot room
4. Vacate room
5. List students
6. List rooms
7. Add complaint
8. Resolve complaint
9. Record payment
10. Generate report
0. Save & exit

Choose: 1

ID: 101

Name: Mehul Vashisth

Phone: 9876543210

Course: AI & Data Science

Year: 1

===== HOSTEL MANAGEMENT SYSTEM =====

Choose: 2

Room no: 101

Capacity: 2

===== HOSTEL MANAGEMENT SYSTEM =====

Choose: 3

Student ID: 101

Room no: 101

===== HOSTEL MANAGEMENT SYSTEM =====

Choose: 10

Students: 1

Rooms: 1

Occupied beds: 1

Complaints: 0

Payments: 0.0

===== HOSTEL MANAGEMENT SYSTEM =====

Choose: 0

Saved successfully.

6. CONCLUSION & FUTURE SCOPE

The **Hostel Management System** successfully demonstrates key Core Java principles, including Object-Oriented Design, Exception Handling, File I/O Streams, and the Java Collections Framework (using Streams & Lambdas).

Future Enhancements:

- **Graphical User Interface (GUI):** Transitioning from standard console interface to JavaFX or Swing.
- **Database Integration:** Replacing file serialization with relational databases (e.g., MySQL or PostgreSQL) using JDBC or JPA/Hibernate.
- **Role-Based Access Control (RBAC):** Providing separate portals for Students, Wardens, and Finance personnel.